The distinction between these three levels can be very fuzzy. For example, most complied languages allow you to directly manipulate the hardware. Likewise, a high-level language with a well stocked library of DSP functions is very close to being an applications package. The point of these three catagories is understand what you are manipulating: (1) hardware, (2) abstract variables, or (3) entire procedures and algorithms.

There is also another important concept behind these classifications. When you use a high-level language, you are relying on the programmer who wrote the compiler to understand the best techniques for hardware manipulation. Similarly, when you use an applications package, you are relying on the programmer who wrote the package to understand the best DSP techniques. Here's the rub: these programmers have never seen the particular problem you are dealing with. Therefore, they cannot always provide you with an optimal solution. As you operate on a *higher* level, expect that the final machine code will be *less* efficient in terms of memory usage, speed, and precision.

Which programming language should you use? That depends on *who* you are and *what* you plan to do. Most computer scientists and programmers use C (or the more advanced C++). Power, flexibility, modularity; C has it all. C is so popular, the question becomes: Why would anyone program their DSP application in something other than C? Three answers come to mind. First, DSP has grown so rapidly that some organizations and individuals are stuck in the mode of other languages, such as FORTRAN and PASCAL. This is especially true of military and government agencies that are notoriously slow to change. Second, some applications require the utmost efficiency, only achievable by assembly programming. This falls into the category of "a little more speed for a lot more work." Third, C is not an especially easy language to master, especially for part time programmers. This includes a wide range of engineers and scientists who occasionally need DSP techniques to assist in their research or design activities. This group often turns to BASIC because of its simplicity.

Why was BASIC chosen for this book? This book is about *algorithms*, not programming style. You should be concentrating on DSP techniques, and not be distracted by the quirks of a particular language. For instance, all the programs in this book have *line numbers*. This makes it easy to describe how the program operates: "line 100 does such-and-such, line 110 does this-and that," etc. Of course, you will probably never use line numbers in your actual programs. The point is, *learning DSP* has different requirements than *using DSP*. There are many books on the market that provide exquisite source code for DSP algorithms. If you are simply looking for prewritten code to copy into your program, you are in the wrong place.

Comparing the execution speed of hardware or software is a thankless task; no matter what the result, the loser will cry that the match was unfair! Programmers who like high-level languages (such as traditional computer scientists), will argue that assembly is only 50% faster than compiled code, but five times more trouble. Those who like assembly (typically, scientists and